

AWS Reliability & Troubleshooting Handbook

EC2 • S3 • VPC • IAM • RDS • Lambda • CloudWatch

A practical SRE-oriented guide that combines the supplied AWS reference sheets with deeper operational explanations, common failure modes, detection methods, mitigation steps, and STAR-style incident narratives (Scenario, Task, Action, Resolve).

How to use this handbook

- Start with symptoms, then identify the failing layer: application, identity, network, compute, storage, database, or observability.
- Use the service-specific detection checklist before changing infrastructure.
- Prefer evidence from logs, metrics, events, CloudTrail, flow logs, and direct connectivity tests.
- During incidents, mitigate user impact first; then perform root-cause analysis and prevention work.
- Use the STAR sections as templates for manager updates, interview answers, postmortems, and incident summaries.

Part I - Original AWS Reference Sheets

The following pages preserve the reference material you provided.

CloudWatch - Reference Sheet



AWS CloudWatch

Monitoring and Observability Service

CloudWatch =
Monitor logs,
metrics, alarms,
dashboards

1. DEFINITION

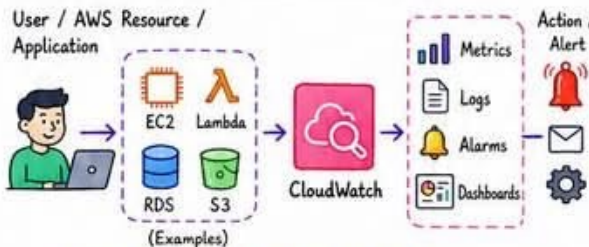
- AWS CloudWatch is a monitoring and observability service in AWS.
- It collects metrics, logs, and events from AWS resources and applications.
- It helps monitor performance, set alarms, visualize data, and automate responses.
- Main idea: observe, alert, and act. ★

2. KEY FEATURES

- ✓ Metrics monitoring
- ✓ Log collection and analysis
- ✓ Alarms and notifications
- ✓ Dashboards for visualization
- ✓ Event-driven automation
- ✓ Works with many AWS services
- ✓ Near real-time monitoring
- ✓ Custom metrics support



3. HOW CLOUDWATCH WORKS ?



CloudWatch collects data and helps you monitor system health. ❤️

4. MAIN COMPONENTS

- **Metrics** = numerical performance data
- **Logs** = record of events/messages
- **Alarms** = trigger action when threshold is crossed
- **Dashboards** = visual monitoring panels
- **Events / EventBridge integration** = respond to state changes/events
- **Log Groups** = container for log streams
- **Log Streams** = sequence of log events
- **Namespace** = container for metrics
- **Dimensions** = name-value pairs to filter metrics

5. TYPES OF MONITORING DATA

Type	Meaning	Example
Metrics	numeric values	CPU utilization
Logs	text records	app logs, Lambda logs
Events	state changes	EC2 state change
Dashboards	visual display	monitoring panel
Alarms	threshold-based alerts	CPU > 80%

6. IMPORTANT FORMULAS / RULES

Formulas

$$\text{Availability (\%)} = \left(\frac{\text{Uptime}}{\text{Total Time}} \right) \times 100$$

$$\text{Error Rate (\%)} = \left(\frac{\text{Errors}}{\text{Total Requests}} \right) \times 100$$

$$\text{Average Latency} = \frac{\text{Total Response Time}}{\text{Number of Requests}}$$

Alarm Rule Example

Trigger when CPUUtilization > 80% for 5 minutes

Exam Note

- ★ CloudWatch itself does monitoring; SNS is commonly used for notification delivery.

7. REAL-LIFE EXAMPLES / USE CASES

- Monitor EC2 CPU and memory
- Track Lambda errors and duration
- Watch RDS performance
- Centralize application logs
- Set billing or usage alerts
- Build dashboards for operations teams
- Detect failures and notify admins
- Trigger Auto Scaling or automation

9. EXAM TIPS & TRICKS

- ★ CloudWatch = performance + monitoring
- ★ CloudTrail = API history / audit trail
- ★ Metrics are numbers, Logs are text
- ★ Alarms watch metrics and trigger actions
- ★ Dashboards give one-screen visibility
- ★ Remember: CloudWatch does not replace CloudTrail

Mnemonic
M - L - A - D
M = Metrics
L = Logs
A = Alarms
D = Dashboards

8. CLOUDWATCH vs CLOUDTRAIL (Difference)

Feature	CloudWatch	CloudTrail
Purpose	Monitoring performance and observability	Auditing API activity and governance
Data Type	Metrics, Logs, Alarms, Events, Dashboards	API calls, Account activity logs
Best For	Operational visibility, alerts, troubleshooting	Security, compliance, audit trail
Example	CPU utilization graph	Who created an EC2 instance?

10. QUICK REVISION (LAST MINUTE)

- ✓ CloudWatch monitors AWS resources and apps
- ✓ Works with metrics, logs, alarms, dashboards
- ✓ Used for visibility, alerting, and automation
- ✓ Common with EC2, Lambda, RDS, S3
- ✓ Alarms can notify through SNS
- ✓ Great for operations and troubleshooting



AWS CloudWatch = Monitor, visualize, alert, and automate in the cloud. ❤️

Lambda - Reference Sheet



AWS Lambda

Serverless Compute Service



Lambda =
Run code
without
managing servers



1. DEFINITION

- AWS Lambda is a serverless compute service by AWS.
- It runs your code in response to events.
- You do not manage servers.
- AWS handles scaling, patching, and availability.
- Main idea: event-driven compute. ★

2. KEY FEATURES

- ✓ Serverless
- ✓ Automatic scaling
- ✓ Pay only for requests + execution time
- ✓ Supports many languages
- ✓ Event-driven
- ✓ Integrates with many AWS services
- ✓ Built-in high availability
- ✓ Good for short-lived tasks





3. HOW LAMBDA WORKS?

User / Event Source → AWS Lambda Function → Runs Code → Response / Output

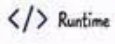
Example Event Sources: API Gateway, S3, DynamoDB, EventBridge, SQS

- An event triggers the function.
- Lambda runs the function code.
- Output is returned or sent to another service.
- Scales automatically with incoming events.



4. MAIN COMPONENTS

- Function** = your code
- Trigger** = event source that starts the function
- Runtime** = language environment (Python, Node.js, Java, etc.)
- Handler** = method Lambda runs first
- Layers** = shared libraries / dependencies
- Execution Role** = IAM role with permissions
- Environment Variables** = config values
- Memory** = affects CPU power too
- Timeout** = max run time



5. INVOCATION TYPES / TRIGGERS

Type	Meaning	Example
Synchronous	caller waits for response	API Gateway
Asynchronous	event queued, response not immediate	S3, EventBridge
Poll-based	Lambda polls source	SQS, Kinesis, DynamoDB Streams

Common Triggers: API Gateway, S3, SQS, SNS, EventBridge, DynamoDB Streams

6. IMPORTANT FORMULAS / RULES

Total Cost = Request Charges + Compute Charges
Compute Charges depend on Memory allocated × Duration
Concurrency = Requests per second × Average Duration (seconds)
Timeout max = 15 minutes

Example:
If 100 requests/sec and avg duration = 2 sec,
Concurrency = $100 \times 2 = 200$
★ More memory usually gives more CPU.
★ Billing is based on request count and execution duration.



7. SECURITY & BEST PRACTICES

- Use least-privilege IAM role
- Do not hardcode secrets
- Use AWS Secrets Manager or Parameter Store
- Set correct timeout and memory
- Keep functions small and focused
- Reuse connections when possible
- Use DLQ or retry handling when needed
- Monitor with CloudWatch
- Use VPC only when required



8. REAL-LIFE EXAMPLES / USE CASES

- Resize images uploaded to S3
- Build serverless APIs with API Gateway
- Process files or logs automatically
- Send notifications from events
- Run scheduled tasks with EventBridge
- ETL / lightweight data processing
- Chatbot or webhook backend
- DynamoDB stream processing



9. LAMBDA VS EC2 (DIFFERENCE)

Feature	Lambda	EC2
Type	Serverless	Virtual server
Server management	No	Yes
Scaling	Automatic	Manual / Auto Scaling
Billing	Per request + duration	Per instance time
Best for	Event-driven tasks, APIs	Long-running apps, full control
Max runtime	15 min	No fixed Lambda-style limit

10. EXAM TIPS & TRICKS

- Remember: Lambda = event-driven + serverless
- Lambda is stateless between invocations
- Temporary storage is in /tmp
- Timeout max is 15 minutes
- Execution role gives AWS permissions
- Layers help share dependencies
- Cold start = startup delay for a new environment
- Good for microservices and automation



Mnemonic: T-H-R-I-V-E
T = Trigger
H = Handler
R = Runtime
I = IAM Role
V = Variables
E = Execution time

11. DIAGRAM / GRAPH EXPLANATION

S3 Upload → Lambda Function → Resize Image → Save to Output Bucket

- Image uploaded to S3 input bucket.
- S3 event triggers the Lambda function.
- Lambda resizes the image and saves it to output bucket.



12. QUICK REVISION (LAST MINUTE)

- ✓ Lambda = serverless compute
- ✓ Runs code on events
- ✓ Auto scales
- ✓ Pay per request + duration
- ✓ Max timeout = 15 min
- ✓ Memory affects CPU
- ✓ Uses IAM execution role
- ✓ Common triggers: API Gateway, S3, SQS, EventBridge
- ✓ Stateless between runs
- ✓ Temporary storage: /tmp
- ✓ Cold start exists
- ✓ Great for event-driven workloads



★ AWS Lambda = Scalable, event-driven, serverless compute in the cloud. ♥ ★

RDS - Reference Sheet



AWS RDS

Amazon Relational Database Service



RDS = Reliable Relational Database Service

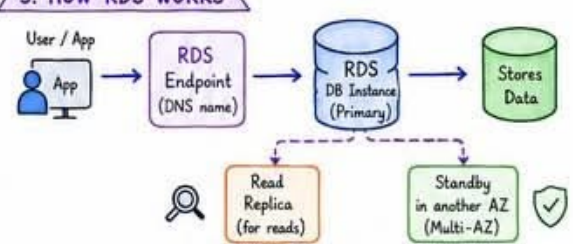
1. DEFINITION

- AWS RDS is a managed relational database service in AWS.
- It helps you set up, operate, and scale relational databases.
- AWS handles backups, patching, monitoring, and high availability.
- Main idea: managed relational databases in the cloud.

2. KEY FEATURES

- Managed service
- Automatic backups
- Easy scaling of compute and storage
- Multi-AZ high availability
- Read Replicas for read scaling
- Monitoring with CloudWatch
- Security with IAM, VPC, KMS, SSL
- Supports multiple database engines

3. HOW RDS WORKS



- App connects using endpoint, port, username/password.
- Data is stored in the DB instance.
- Multi-AZ keeps a standby copy for failover.
- Read Replicas improve read performance.

4. MAIN COMPONENTS

- DB Instance = compute + memory for the database engine
- DB Engine = MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, Aurora
- Storage = SSD-backed database storage
- Endpoint = DNS name used by app to connect
- Parameter Group = DB engine configuration
- Option Group = extra engine-specific features
- Snapshot = manual backup copy
- Automated Backup = scheduled backup managed by RDS
- Security Group = controls network access
- DB Subnet Group = tells RDS which subnets it can use

5. DATABASE ENGINES / TYPES

Engine	Best For	Notes
MySQL	web apps	popular open-source
PostgreSQL	advanced SQL apps	powerful features
MariaDB	MySQL-compatible workloads	open-source fork
Oracle	enterprise systems	licensed engine
SQL Server	Microsoft stack apps	enterprise/business apps
Aurora	high-performance cloud-native relational DB	MySQL/PostgreSQL compatible

6. DEPLOYMENT / AVAILABILITY OPTIONS

Option	Description
Single-AZ	One primary DB in one AZ. Lower cost, no standby HA.
Multi-AZ	Primary + standby in another AZ. High availability, automatic failover.
Read Replica	Asynchronous copy for read scaling and reporting.

Exam Note: Multi-AZ = HA, Read Replica = performance/read scaling

7. STORAGE TYPES

Storage Type	Description
General Purpose SSD (gp)	Balanced cost/performance
Provisioned IOPS SSD	High I/O performance, mission-critical workloads
Magnetic / legacy	Older, low performance option

Note: Storage can scale. Some engines support storage autoscaling.

8. BACKUP & RECOVERY

- Automated backups allow point-in-time restore.
- Backup retention: 0 to 35 days.
- Manual snapshots are kept until you delete them.
- Point-in-time recovery restores to a chosen time within retention period.
- Snapshots can be copied across regions/accounts.
- Multi-AZ improves recovery readiness.

9. SECURITY

- Use Security Groups to allow DB traffic.
- Use IAM for management permissions.
- Encrypt data at rest with KMS.
- Use SSL/TLS for data in transit.
- Place RDS in private subnets when possible.
- Use strong passwords / Secrets Manager.
- Best practice: least privilege access.

10. IMPORTANT FORMULAS / RULES

Availability (%) = (Uptime / Total Time) × 100

Storage Cost = Storage Used × Price per GB

Total RDS Cost = Instance Cost + Storage Cost + Backup/IO Cost

- Multi-AZ = high availability, not read scaling.
- Read Replica = read scaling, can be promoted.
- RDS is for relational databases, DynamoDB is NoSQL.
- Automated backup window and maintenance window are different.

11. DIFFERENCE TABLE

Feature	RDS	DynamoDB	EC2-hosted DB
Type	relational	NoSQL	self-managed relational/any DB
Management	managed	fully managed	you manage everything
Scaling	vertical + read replicas	massive managed scaling	manual
Backups	automated	managed	self-managed
Best for	SQL apps	key-value / document	full control / custom setup

12. REAL-LIFE EXAMPLES / USE CASES

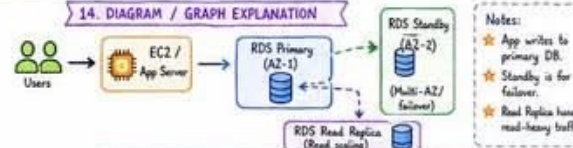
- E-commerce app storing users, orders, payments
- School / student management system
- Banking / business reporting database
- CMS or blog backend
- SaaS app with transactional data
- Analytics / reporting with read replicas

13. EXAM TIPS / TRICKS

- Remember: RDS = relational.
- Multi-AZ = availability.
- Read Replica = read performance.
- Snapshot = manual backup.
- Automated backup = PITR support.
- Security Group is stateful.
- RDS usually sits in private subnet.

Memory Trick: **M-R-S** for Multi-AZ, Read Replica, Resilience, Speed.

14. DIAGRAM / GRAPH EXPLANATION



15. QUICK REVISION (LAST MINUTE)

- RDS = managed relational DB service
- Supports MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, Aurora
- Endpoint used to connect
- Multi-AZ = HA
- Read Replica = scaling reads
- Backups + snapshots protect data
- Use encryption, private subnets, security groups
- Easy to operate and scale
- Great for relational, transactional workloads

♥ AWS RDS = Managed, secure, scalable relational databases in the cloud. ♥

IAM - Reference Sheet



AWS IAM

Identity and Access Management



1. DEFINITION



- AWS IAM = Identity and Access Management.
- It helps you securely manage who can sign in and what they can do in AWS.
- Used to control authentication and authorization.
- IAM is a global service.
- Main goal: **least-privilege access**.

2. KEY FEATURES



- ✓ Secure access control
- ✓ Fine-grained permissions
- ✓ Supports users, groups, roles, policies
- ✓ MFA support
- ✓ Temporary credentials with roles
- ✓ Works across almost all AWS services
- ✓ Free service (you pay for other AWS resources, not IAM itself)

3. HOW IAM WORKS



- Authentication = Who are you?
- Authorization = What can you do?
- Access is decided by attached policies.

4. MAIN COMPONENTS

- User** = Individual identity for one person or app.
- Group** = Collection of users with same permissions.
- Role** = Identity with permissions that can be assumed temporarily.
- Policy** = JSON document that defines permissions.
- Root User** = Full account owner; avoid daily use.
- MFA** = Extra security layer.
- Access Key** = Programmatic access for CLI/API.
- ARN** = Amazon Resource Name, unique resource identifier.

5. TYPES OF POLICIES

Policy Type	Meaning	Example Use
Identity-based policy	Attached to user/group/role	Give S3 read access
Resource-based policy	Attached to resource	S3 bucket policy
Permissions boundary	Max permission limit	Control developer accounts
SCP (AWS Organizations)	Account-level guardrail	Block certain services in child accounts
Session policy	Temporary session restriction	Restrict assumed role session

6. POLICY STRUCTURE

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::my-bucket/*",
    "Condition": {
      "StringEquals": {"s3:prefix": "home/"}
    }
  }]
}
```

- **Version** = Policy language version
- **Effect** = Allow or Deny
- **Action** = What actions are allowed/denied
- **Resource** = Which resources the policy applies to
- **Condition** = Optional rules (IP, time, tags, etc.)

7. EXAM FORMULAS / RULES

Remember:
Explicit Deny
always wins!

- ★ **Effective Permissions** = Allowed permissions - Explicit Denies
- ★ **Explicit Deny > Allow**
- ★ **By default**, all requests are implicitly denied until allowed
- ★ **Least Privilege** = Give only required permissions
- ★ **MFA** = Something you know + something you have

8. AUTHENTICATION METHODS



- Console password
- Access keys (CLI/API)
- MFA
- Federated login / SSO
- AssumeRole / temporary security credentials

9. BEST PRACTICES



- ✓ Do not use root user for daily work
- ✓ Enable MFA, especially for root and admins
- ✓ Give permissions through groups and roles when possible
- ✓ Rotate or avoid long-term access keys
- ✓ Follow least privilege
- ✓ Use IAM roles for EC2/Lambda instead of hardcoding credentials
- ✓ Review CloudTrail logs and IAM Access Analyzer
- ✓ Remove unused users and permissions
- ✓ Use strong password policy

10. REAL-LIFE EXAMPLES



- Developer needs read-only access to S3 → attach read policy
- EC2 instance needs DynamoDB access → assign IAM role to EC2
- Company employees join "Developers" group → inherit same permissions
- Cross-account access → assume role in another AWS account
- Admin account protected with MFA

11. DIFFERENCE TABLE

Item	What it is	Used for	Credentials?
User	Individual identity	People or apps	Yes
Group	Collection of users	Manage permissions at scale	No
Role	Assumable identity	AWS services or cross-account access	Temporary
Policy	JSON permission document	Allow or deny access	No

IAM vs ROOT USER

Aspect	IAM User	Root User
Permissions	Limited, controlled	Full access to everything
Number	Many (you create)	One per AWS account
Best for	Daily tasks, apps	Account setup & recovery
Risk	Lower	Highest (very dangerous)

12. COMMON EXAM POINTS / TRICKS

- ★ IAM is global, not regional
- ★ Policy language is JSON
- ★ Role = temporary access
- ★ Group cannot contain another group
- ★ You cannot attach a password policy to a single user; it applies to the account
- ★ Root user permissions cannot be restricted by IAM policies
- ★ Access keys are for programmatic access
- ★ Explicit deny always wins

Mnemonics / Tricks

"UGRP"
Users, Groups, Roles,
Policies = core IAM
building blocks

"AAA"
Authenticate, Authorize,
Audit

13. DIAGRAM / GRAPH EXPLANATION



- If allowed, request succeeds.
- If denied or not allowed, request fails.

14. QUICK REVISION (LAST MINUTE)



- IAM = Identity and Access Management
- Global service
- Controls authentication and authorization
- Main entities: user, group, role, policy
- Policies are JSON
- Explicit deny beats allow
- Use least privilege and MFA
- Roles provide temporary credentials



★ AWS IAM = Securely manage access to AWS resources. ★

VPC - Reference Sheet



AWS VPC

Amazon Virtual Private Cloud

VPC = Your own virtual network in the AWS Cloud

1. DEFINITION

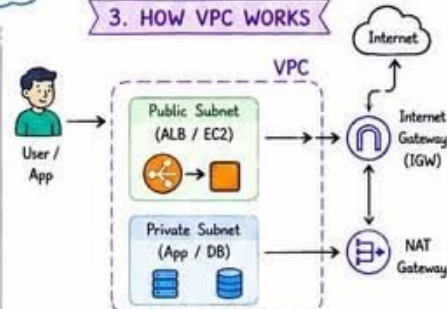
- AWS VPC is a logically isolated virtual network in AWS.
- Lets you launch AWS resources in your own virtual network.
- You control IP ranges, subnets, route tables, and security.
- Main idea: network isolation + connectivity control. ★

2. KEY FEATURES

- ✓ Isolation inside AWS cloud
- ✓ Custom IP range using CIDR
- ✓ Public and private subnets
- ✓ Route control
- ✓ Security Groups and NACLs
- ✓ Internet connectivity via IGW
- ✓ Private outbound internet via NAT Gateway
- ✓ Connect to on-prem with VPN / Direct Connect
- ✓ Scalable and flexible networking



3. HOW VPC WORKS



- Public subnet = has route to IGW
- Private subnet = no direct route to internet ★
- Route tables decide traffic path.

4. MAIN COMPONENTS

- VPC = your virtual network
- CIDR Block = IP address range of VPC
- Subnet = smaller IP range in one AZ
- Route Table = controls traffic direction
- Internet Gateway (IGW) = internet access for public subnets
- NAT Gateway = outbound internet for private subnets

- Security Group = instance-level stateful firewall
- Network ACL (NACL) = subnet-level stateless firewall
- Elastic IP = static public IPv4 for resources (e.g., NAT GW)
- VPC Endpoint = private access to AWS services
- Peering = connect two VPCs
- Transit Gateway = hub to connect many VPCs/networks

5. SUBNET TYPES / NETWORKING BASICS

Feature	Public Subnet	Private Subnet
Internet access	Yes via IGW	No direct internet
Typical resources	ALB, bastion host, web server	app server, DB, internal services
Route to IGW	Yes	No
Common use	internet-facing layer	backend / secure layer

Note: One subnet belongs to one Availability Zone (AZ).

6. IMPORTANT FORMULAS / RULES

Total IPv4 addresses in subnet
= $2^{(32 - \text{prefix})}$

Usable IPv4 addresses in AWS subnet
= $2^{(32 - \text{prefix})} - 5$

CIDR Examples (Total IPs)		
/16	/24	/28
65,536	256	16

Example: /24 subnet

- Total = $2^{(32-24)} = 256$
- Usable = $256 - 5 = 251$

⚡ AWS reserves 5 IPs in each subnet.

7. ROUTING & CONNECTIVITY

- Route table tells where traffic goes.
- Local route enables communication within VPC.
- 0.0.0.0/0 to IGW = public internet route.
- 0.0.0.0/0 to NAT Gateway = private outbound internet.
- VPC Peering connects VPCs privately.
- Transit Gateway connects many VPCs.
- VPN = encrypted tunnel to on-prem.
- Direct Connect = dedicated private connection to AWS.

9. REAL-LIFE EXAMPLES / USE CASES

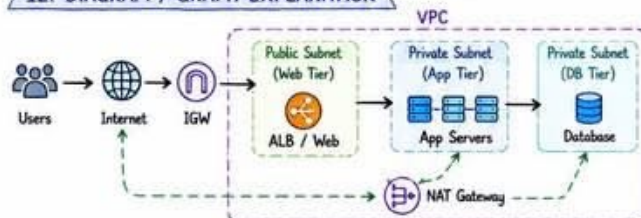
- ✓ Host a 3-tier web app.
- ✓ Put web servers in public subnets and DB in private subnets.
- ✓ Connect office network to AWS with VPN.
- ✓ Build secure internal applications.
- ✓ Separate dev / test / prod environments.
- ✓ Disaster recovery / multi-environment setup.

8. SECURITY

- Security Groups: stateful, allow rules only.
- NACLs: stateless, allow and deny rules.
- Use least privilege. 🛡️

Security Group (SG) vs NACL		
Aspect	Security Group (SG)	NACL
Stateful?	Yes	No
Level	Instance-level	Subnet-level
Rules	Allow only	Allow & Deny

12. DIAGRAM / GRAPH EXPLANATION



11. DIFFERENCE TABLE

Item	Purpose	Internet Access	Typical Use
IGW	Connect VPC to internet	Yes	Public subnets
NAT Gateway	Private subnet outbound internet	Outbound only	Private subnets
VPC Endpoint	Private access to AWS services	No internet needed	S3, DynamoDB, AWS APIs

13. QUICK REVISION (LAST MINUTE)

- ✓ VPC = isolated AWS network
- ✓ Subnet = part of VPC in one AZ
- ✓ Public subnet has route to IGW
- ✓ Private subnet usually uses NAT for outbound internet
- ✓ SG = stateful, NACL = stateless
- ✓ CIDR decides IP range
- ✓ Route table controls path
- ✓ VPC endpoints give private AWS service access

AWS VPC = Control, Connectivity, and Security for your cloud network. ❤️

S3 - Reference Sheet



AWS S3

Simple Storage Service



S3 = Scalable
Secure
Simple
Storage

1. DEFINITION

- Amazon S3 (Simple Storage Service) is an object storage service offered by AWS.
- It stores data as objects inside buckets.
- Highly durable, available, scalable and secure. ★

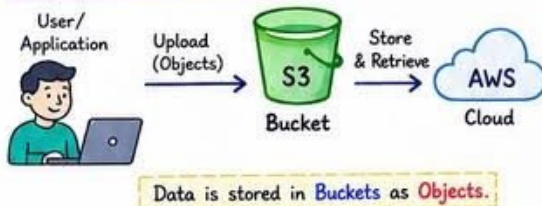


2. KEY FEATURES

- ✓ Unlimited Storage
- ✓ 99.99999999% Durability (11 9's)
- ✓ 99.99% Availability
- ✓ Secure & Private
- ✓ Scalable – from GBs to PBs
- ✓ Cost-effective (Pay for what you use)
- ✓ Works with many AWS services

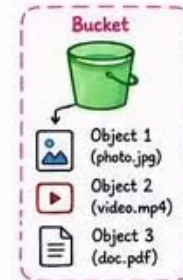


3. HOW S3 WORKS ?



4. MAIN COMPONENTS

- Bucket** – Container to store objects (must be globally unique)
- Object** – Actual file (data + metadata)
- Key** – Unique name of the object inside a bucket
- Region** – Physical location of bucket
- Endpoint** – URL used to access S3



5. S3 STORAGE CLASSES

Storage Class	Use Case	Durability	Cost
S3 Standard	Frequent access	99.99999999%	\$\$\$
S3 Standard-IA (Infrequent Access)	Infrequent access	99.99999999%	\$\$
S3 One Zone-IA	Infrequent, non-critical data	99.99999999%	\$
S3 Glacier Instant Retrieval	Archive, needs fast retrieval	99.99999999%	\$
S3 Glacier Flexible Retrieval (Deep Archive)	Long-term archive	99.99999999%	\$
S3 Glacier Deep Archive	Lowest cost archive	99.99999999%	\$

6. DURABILITY & AVAILABILITY

Durability Formula

$$\text{Durability (\%)} = (1 - P_{\text{loss}}) \times 100$$

Where P_{loss} = Probability of data loss

Example:

$$\text{If } P_{\text{loss}} = 10^{-11}$$

$$\text{Durability} = (1 - 10^{-11}) \times 100 = 99.99999999\% \text{ (11 9's)}$$

Availability Formula

$$\text{Availability (\%)} = (\text{Uptime} / \text{Total Time}) \times 100$$

Example:

$$\text{If Uptime} = 525,600 \text{ min (1 year)}$$

$$\text{Total Time} = 525,600 \text{ min}$$

$$\text{Availability} = (525600 / 525600) \times 100 = 100\%$$

7. USE CASES (REAL LIFE EXAMPLES)

- Website Hosting → Store website images, CSS, JS, HTML
- Backup & Restore → Backup databases, files
- Data Lake → Store big data for analytics
- Media Storage → Store videos, photos, audio files
- Disaster Recovery → Store important data safely



9. EXAM TIPS & TRICKS

- Trick to Remember Storage Classes: "S S O G D"
 - S → Standard
 - S → Standard-IA
 - O → One Zone-IA
 - G → Glacier Instant Retrieval
 - D → Glacier Flexible Retrieval
 - D → Glacier Deep Archive
- Bucket name must be globally unique. ★
- S3 is Object Storage, not for OS or DB directly.
- Higher 9's = Higher Durability

8. S3 vs EBS (Difference)

Feature	S3	EBS
Type	Object Storage	Block Storage
Access	HTTP/HTTPS	Attached to EC2
Scalability	Virtually Unlimited	Limited (per volume)
Use Case	Big data, backup, static files	OS, Database, Applications
Cost	Low (pay per use)	Higher
Durability	11 9's	99.99% (3 9's)

10. QUICK REVISION (LAST MINUTE)

- ✓ S3 = Simple Storage Service (Object Storage)
- ✓ Stores data in Buckets as Objects
- ✓ Durability = 11 9's (99.99999999%)
- ✓ Availability = 99.99%
- ✓ Scalable, Secure, Cost-effective
- ✓ Storage classes help reduce cost
- ✓ Works with many AWS services



S3 = Simple, Scalable, Secure, and Highly Durable Storage in the Cloud ♥

EC2 - Reference Sheet



AWS EC2

Elastic Compute Cloud



1. DEFINITION

- AWS EC2 = Amazon Elastic Compute Cloud.
- It provides resizable virtual servers in the cloud.
- Used to run applications, websites, APIs, and workloads on demand.
- Main idea: scalable compute capacity.



2. KEY FEATURES



- ✓ On-demand virtual machines
- ✓ Scalable up/down
- ✓ Many instance types
- ✓ Pay-as-you-go pricing
- ✓ Secure with IAM + Security Groups
- ✓ Supports EBS, AMIs, Auto Scaling, Load Balancers
- ✓ Global availability through Regions and AZs



3. HOW EC2 WORKS



- Select:
- AMI
 - Instance Type
 - Storage (EBS)
 - Security Group
 - Key Pair



• AMI = template (preconfigured image)

• Security Group = virtual firewall

• Instance Type = hardware choice (CPU, RAM, network)

• EBS = storage for your instance

4. MAIN COMPONENTS

- AMI = Amazon Machine Image, preconfigured template
- Instance = Virtual server running in AWS
- Instance Type = CPU, RAM, storage, network combination
- EBS = Elastic Block Store (persistent storage)
- Security Group = Instance-level firewall (allow rules only)
- Key Pair = Secure login credentials (SSH for Linux)
- Elastic IP = Static public IP-v4 address
- User Data = Startup script executed at first boot
- Region / AZ = Deployment location (Region, Availability Zone)

5. INSTANCE TYPES

Family	Best For	Example
General Purpose	balanced compute/memory	t3, t4g, m5
Compute Optimized	CPU-heavy apps	c5, c6i
Memory Optimized	in-memory DB / caching	r5, x1
Storage Optimized	high local storage / IOPS	i3, d3
Accelerated Computing	GPU / ML / graphics	p4, g5



QUICK TRICK:

T = tiny burstable, M = mixed,
C = compute, R = RAM, P/G = GPU

6. PRICING MODELS

Model	Meaning	Best Use
On-Demand	pay per use	short-term / unpredictable
Reserved Instances / Savings Plans	commit for lower cost	steady workloads
Spot Instances	unused capacity, very cheap	fault-tolerant jobs
Dedicated Hosts / Instances	isolated hardware	compliance / licensing

7. IMPORTANT FORMULAS / RULES



- Cost Formula: Total Cost = Hourly Rate × Hours Used + Storage Charges + Data Transfer Charges
- Availability Formula: Availability (%) = (Uptime / Total Time) × 100
- Scaling Rule: Vertical scaling = bigger instance
Horizontal scaling = more instances
- Exam Rule: Security Group = allow rules only;
NACL = allow + deny
- EBS persists independently of instance.
(unless configured otherwise).



8. STORAGE WITH EC2

- EBS = persistent block storage
- Instance Store = temporary local storage
- EFS = shared file storage

Storage	Persistent?	Use Case
EBS	Yes	OS disk / databases
Instance Store	No	temporary cache / buffers
EFS	Yes	shared Linux file system



9. SECURITY



- IAM roles for AWS permissions
- Security Groups control inbound/outbound traffic
- NACL works at subnet level
- Key pairs for SSH access (Linux)
- Use SSM Session Manager as modern access option
- Encrypt volumes with KMS
- Use least privilege



10. REAL-LIFE EXAMPLES / USE CASES



- Host a website or web app
- Run a backend API server
- Create a dev/test machine
- Big data / batch processing
- Game servers
- ML or GPU workloads
- Disaster recovery servers

11. EC2 vs LAMBDA vs ECS

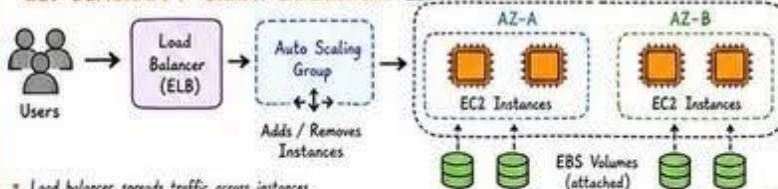
Service	Best For	Server Management	Billing Style
EC2	full control VMs	Yes	per instance usage
Lambda	event-driven code	No	per request + duration
ECS	containers	Partial	container/instance based

12. BEST PRACTICES / EXAM TIPS



- ✓ Choose the right instance family
- ✓ Put apps in multiple AZs for high availability
- ✓ Use Auto Scaling for variable traffic
- ✓ Use ELB to distribute traffic
- ✓ Stop unused dev instances to save cost
- ✓ Use AMIs for fast repeat deployment
- ✓ Monitor with CloudWatch
- ✓ Back up using snapshots
- ✓ Never open port 22/3389 to the world unless necessary
- ✓ Remember: EC2 gives most control, but also most management responsibility

13. DIAGRAM / GRAPH EXPLANATION



- Load balancer spreads traffic across instances.
- Auto Scaling adds or removes instances based on demand.
- Multi-AZ deployment improves availability and fault tolerance.



14. QUICK REVISION (LAST MINUTE)



- ✓ EC2 = virtual server service
- ✓ AMI = template
- ✓ Instance Type = compute size
- ✓ EBS = persistent storage
- ✓ Security Group = firewall
- ✓ Key Pair = login access
- ✓ On-Demand / Reserved / Spot = pricing options
- ✓ Auto Scaling + ELB = scalability + HA
- ✓ Use CloudWatch for monitoring
- ✓ EC2 is best when you need OS-level control



★ AWS EC2 = Scalable, secure, and cost-effective compute in the cloud ★

Part II - Reliability Troubleshooting Method

A reliable incident workflow avoids random changes. The goal is to move from symptom to evidence, isolate the failing layer, mitigate impact, verify recovery, and then prevent recurrence.

- 1. Define the symptom** - What is failing? Which users, regions, endpoints, or workloads are affected? When did it start?
- 2. Check recent changes** - Deployments, configuration, IAM, DNS, certificates, route changes, scaling events, maintenance.
- 3. Establish blast radius** - Single pod/instance, one AZ, one service, one account, or global?
- 4. Verify dependencies** - DNS -> TCP -> TLS -> authentication -> application response.
- 5. Use observability** - Metrics, logs, traces, CloudTrail, VPC Flow Logs, RDS events, ASG history.
- 6. Mitigate** - Rollback, fail over, restart only when justified, add capacity, disable bad path, restore permissions.
- 7. Verify** - Test from the real workload path and confirm user-facing metrics recover.
- 8. Prevent recurrence** - Runbook, alarm, test, policy guardrail, capacity margin, deployment validation, postmortem action.

Fast connectivity isolation

- DNS fails -> resolver / hosted zone / CoreDNS / forwarding problem.
- DNS works, TCP times out -> routing / SG / NACL / firewall / return-path problem.
- TCP connects, TLS fails -> certificate / SNI / protocol problem.
- TLS works, HTTP 401/403 -> identity / policy / token / application authorization problem.
- HTTP 5xx -> application or dependency failure; correlate with logs and downstream health.

Part III - CloudWatch: Reliability, Failures, Detection & Fixes

CloudWatch is the operational visibility layer for AWS workloads. It collects metrics, logs, events, alarms, and dashboard data so operators can detect degradation, investigate incidents, and automate responses.

What to monitor

- Alarm state changes
- CloudWatch Logs Insights queries
- Metric graphs and anomaly bands
- Contributor Insights / dashboards
- EventBridge events for resource state changes

Common failures and how to fix them

Failure	Symptoms	How to detect	Fix / Mitigation
Missing or delayed metrics	Dashboards show gaps or alarms never trigger.	Check namespace, dimensions, agent status, metric publication interval, and IAM permissions.	Fix agent/configuration, metric namespace/dimensions, or permissions; verify with a fresh datapoint.
Noisy or ineffective alarms	Too many false positives or important events are missed.	Review threshold, evaluation periods, missing-data behavior, and metric math.	Tune thresholds, use anomaly detection/composite alarms where appropriate, and align alarms to user impact.
Log ingestion failure	Application logs stop appearing in a log group.	Check CloudWatch agent/Fluent Bit, log group/stream permissions, network path, and disk pressure.	Restore agent, permissions, or network egress; confirm new events arrive.
Dashboard looks healthy while users fail	Infra metrics are green but service is unavailable.	Correlate infra metrics with application errors, latency, synthetic checks, and downstream dependencies.	Add service-level SLIs such as success rate, p95 latency, queue age, and dependency health.

Useful checks / commands

```
aws cloudwatch describe-alarms
```

```
aws cloudwatch get-metric-data ...
```

```
aws logs tail <log-group> --follow
```

```
aws logs start-query --log-group-name <group> --start-time ... --end-time ... --query-string  
'fields @timestamp,@message | sort @timestamp desc | limit 50'
```

STAR reliability example

Scenario: Production API latency suddenly increased and user requests began timing out.

Task: Identify whether the issue was compute, dependency, or application related and restore service quickly.

Action: Used CloudWatch alarms and dashboards to correlate p95 latency with CPU, 5xx rate, dependency errors, and recent deployment events; queried logs for the dominant exception; isolated the failing dependency; mitigated by routing traffic away/restarting unhealthy capacity and opening a follow-up reliability action.

Resolve: Service recovered, the alert was tuned around user-impact metrics, and a dashboard/runbook was added to reduce future MTTR.

Part III - Lambda: Reliability, Failures, Detection & Fixes

Lambda runs event-driven code without server management. Reliability depends on concurrency, timeout, memory, event source configuration, downstream dependencies, retry behavior, and idempotency.

What to monitor

- Errors
- Throttles
- Duration / p95
- ConcurrentExecutions
- IteratorAge
- DLQ depth / event age

Common failures and how to fix them

Failure	Symptoms	How to detect	Fix / Mitigation
Timeouts	Invocations hit configured timeout or upstream caller receives timeout.	Check duration, timeout, network calls, cold starts, downstream latency, and VPC networking.	Increase timeout only if justified; optimize code, reuse connections, fix dependency latency, or remove unnecessary VPC attachment.
Throttling	429/TooManyRequests, rising iterator age, delayed events.	Check account/function concurrency and reserved concurrency.	Raise concurrency where safe, reduce burst pressure, add buffering, or optimize duration.
Permission denied	AccessDenied for S3/DynamoDB/KMS/etc.	Inspect execution role, resource policy, KMS policy, and SCPs.	Add least-privilege permissions and verify with CloudTrail.
Poison event / retry storm	Repeated failures, growing DLQ or queue age.	Inspect failed event payloads and retry settings.	Use DLQ/on-failure destinations, validate input, make handlers idempotent, and quarantine bad events.
Cold-start latency	First requests are much slower than steady state.	Compare Init Duration and Duration.	Reduce package size, initialize lazily, use provisioned concurrency where justified.

Useful checks / commands

```
aws lambda get-function-configuration --function-name <fn>
aws logs tail /aws/lambda/<fn> --follow
aws lambda get-function-concurrency --function-name <fn>
aws cloudwatch get-metric-data ...
```

STAR reliability example

Scenario: A queue-backed Lambda started falling behind and customer jobs were delayed.

Task: Restore processing throughput without losing messages.

Action: Checked CloudWatch for throttles, duration, errors, and SQS age; found long execution duration consuming concurrency. Reduced a slow downstream call path, increased safe reserved concurrency, and enabled

DLQ monitoring.

Resolve: Backlog drained, processing latency returned to normal, and alarms were added on queue age and Lambda throttles.

Part III - RDS: Reliability, Failures, Detection & Fixes

RDS reliability centers on connectivity, storage, compute, query performance, failover, backups, and replication. A healthy database needs both infrastructure health and workload-aware monitoring.

What to monitor

- CPUUtilization
- DatabaseConnections
- FreeStorageSpace
- Read/WriteLatency
- Read/WriteIOPS
- ReplicaLag
- Performance Insights

Common failures and how to fix them

Failure	Symptoms	How to detect	Fix / Mitigation
Connection exhaustion	Applications fail with too many connections or timeouts.	Check DatabaseConnections, max_connections, pool settings, idle sessions.	Fix pooling/leaks, reduce idle connections, use RDS Proxy if appropriate.
High CPU / slow queries	Latency rises, CPU remains high, DB load spikes.	Use Performance Insights, slow query logs, wait events.	Tune queries/indexes, reduce hot queries, scale instance if capacity is truly insufficient.
Storage full / IOPS pressure	FreeStorageSpace near zero, latency spikes.	Check storage metrics, burst balance, write volume.	Increase storage, enable autoscaling, reduce excessive writes, choose suitable storage class.
Failover event	Brief disconnects or endpoint reconnects.	Check RDS events and Multi-AZ state.	Ensure application retries/DNS caching are appropriate; validate Multi-AZ.
Replica lag	Stale reads or DR risk.	Check ReplicaLag and write load.	Reduce write pressure, size replica appropriately, avoid heavy replica queries.

Useful checks / commands

```
aws rds describe-db-instances --db-instance-identifier <db>
aws rds describe-events --source-type db-instance --duration 60
aws cloudwatch get-metric-data ...
nslookup <rds-endpoint>; nc -vz <rds-endpoint> <port>
```

STAR reliability example

Scenario: A production application began returning DB timeout errors during peak traffic.

Task: Determine whether the failure was network, DB capacity, or application connection handling.

Action: Validated endpoint DNS/TCP, checked RDS connections/CPU/storage, and used Performance Insights. Found connection saturation caused by an application pool leak. Rolled out a pool configuration fix and temporarily increased headroom.

Resolve: Timeouts stopped, connection count stabilized, and alarms/runbook checks were added for saturation.

Part III - IAM: Reliability, Failures, Detection & Fixes

IAM controls authentication and authorization across AWS. Reliability incidents often present as sudden AccessDenied failures after policy, role, trust-policy, KMS, SCP, or credential changes.

What to monitor

- CloudTrail denied API calls
- STS caller identity
- IAM policy simulator
- Access Analyzer
- application error logs

Common failures and how to fix them

Failure	Symptoms	How to detect	Fix / Mitigation
AccessDenied	Workload suddenly cannot call an AWS API.	Check identity policy, resource policy, permission boundary, SCP, session policy, explicit deny.	Correct the specific missing permission or deny while preserving least privilege.
AssumeRole failure	STS AssumeRole returns access denied.	Inspect role trust policy and caller permissions.	Fix trust relationship, principal, conditions, external ID, or caller sts:AssumeRole.
Expired credentials	Signature expired / invalid token.	Check credential source and refresh mechanism.	Use roles instead of static keys and restore credential refresh.
KMS denied	S3/Secrets Manager access fails despite service permission.	Check key policy and kms:Decrypt permission.	Grant least-privilege key usage to the correct role.

Useful checks / commands

```
aws sts get-caller-identity
```

```
aws iam get-role --role-name <role>
```

```
aws iam simulate-principal-policy --policy-source-arn <arn> --action-names <action>
```

```
aws cloudtrail lookup-events --lookup-attributes AttributeKey=EventName,AttributeValue=AssumeRole
```

STAR reliability example

Scenario: A pod deployment lost access to an S3 bucket immediately after an IAM change.

Task: Restore access while determining whether the issue was pod identity, role trust, or bucket/KMS policy.

Action: Ran sts get-caller-identity from the workload, compared the active role with the expected role, inspected CloudTrail denies and policies, then corrected the missing permission/trust relationship.

Resolve: S3 access was restored and a deployment validation check was added to catch identity regressions before production rollout.

Part III - VPC: Reliability, Failures, Detection & Fixes

VPC provides the network boundary and routing fabric for AWS workloads. Reliability failures are often caused by route tables, security groups, NACLs, DNS, NAT/IGW, peering/TGW, or cross-account routing.

What to monitor

- VPC Flow Logs
- Reachability Analyzer
- Route tables
- SG/NACL rules
- Route 53 resolver logs
- nc/curl/nslookup from workload

Common failures and how to fix them

Failure	Symptoms	How to detect	Fix / Mitigation
DNS resolution failure	Hostnames do not resolve from workloads.	Check VPC DNS settings, Route 53 resolver/private zones, CoreDNS for EKS, and conditional forwarding.	Attach/correct private hosted zone or resolver rule; verify from workload.
TCP timeout	DNS resolves but connection hangs.	Check route table, SG, NACL, TGW/peering routes, target firewall.	Correct missing route/rule and validate return path.
Connection refused	Immediate TCP refusal.	Network path likely works; service/listener is not accepting connection.	Check target listener, port, service health, and target host firewall.
Asymmetric routing	Intermittent or one-way failures.	Inspect forward and return routes, NAT/TGW path.	Make routes symmetric and ensure stateful devices see both directions.
NAT exhaustion	Intermittent outbound failures at scale.	Check NAT metrics/port usage and destination concentration.	Add NAT capacity, distribute traffic, or use private endpoints.

Useful checks / commands

```
aws ec2 describe-route-tables --filters Name=vpc-id,Values=<vpc>
aws ec2 describe-security-groups --group-ids <sg>
aws ec2 describe-network-acls --filters Name=vpc-id,Values=<vpc>
nslookup <host>; nc -vz <host> <port>; curl -vk https://<host>
```

STAR reliability example

Scenario: An EKS workload could resolve an internal hostname but could not connect to the service.

Task: Determine whether the break was AWS routing, NACL/SG policy, or a Cisco-side firewall/allow-list.

Action: Tested DNS and TCP from the pod, validated Route 53 and VPC routes, checked NACL/SG egress and return path, then used flow evidence to show AWS traffic was leaving correctly. Coordinated the peer-side allow-list update.

Resolve: Connectivity was restored and the endpoint was added to a repeatable egress validation checklist.

Part III - S3: Reliability, Failures, Detection & Fixes

S3 is highly durable object storage, but application reliability still depends on permissions, KMS, endpoint/DNS access, bucket policies, object keys, lifecycle, and event-notification configuration.

What to monitor

- S3 request metrics
- CloudTrail data events
- server access logs
- application errors
- Storage Lens

Common failures and how to fix them

Failure	Symptoms	How to detect	Fix / Mitigation
403 AccessDenied	Application cannot read/write object.	Check IAM role, bucket policy, KMS key policy, object ownership, VPC endpoint policy.	Fix least-privilege policy path and verify caller identity.
404 / wrong key	Expected object is missing.	Check exact bucket/key, prefix, region, application path construction.	Correct key generation or deployment artifact path.
Upload/download timeout	Requests hang or retry.	Check VPC endpoint/NAT/network path and DNS.	Restore route or endpoint access; verify endpoint policy.
Event notification not firing	Lambda/SQS/SNS not triggered.	Check notification config, prefix/suffix filters, destination permissions.	Correct notification rule and destination resource policy.
Unexpected cost	Request/storage charges spike.	Inspect access patterns, storage class, lifecycle, transfer.	Add lifecycle policies, reduce small-request storms, use correct storage class.

Useful checks / commands

```
aws s3api head-bucket --bucket <bucket>
aws s3api head-object --bucket <bucket> --key <key>
aws s3api get-bucket-policy --bucket <bucket>
aws s3api get-bucket-notification-configuration --bucket <bucket>
```

STAR reliability example

Scenario: A Kubernetes workload failed to write compute logs to S3 after a new deployment.

Task: Identify whether the issue was S3, networking, or workload identity.

Action: Verified pod identity with `sts get-caller-identity`, tested bucket access, inspected bucket/KMS policies, and checked VPC endpoint access. Found the workload role lacked the required object-write permission.

Resolve: Updated the role policy, validated writes from the pod, and added an integration test for future deployments.

Part III - EC2: Reliability, Failures, Detection & Fixes

EC2 reliability depends on instance health, capacity, storage, networking, autoscaling, boot configuration, and application process health. Infrastructure health and service health must be evaluated separately.

What to monitor

- EC2 status checks
- CloudWatch CPU/network/disk metrics
- SSM/console logs
- ALB target health
- Auto Scaling activity history

Common failures and how to fix them

Failure	Symptoms	How to detect	Fix / Mitigation
Instance status check failure	EC2 status check fails or instance unreachable.	Check system vs instance status checks, console output, events.	Recover/reboot/replace instance; investigate OS/filesystem/kernel issues.
High CPU	Latency and load rise.	Check CloudWatch CPU, load average, process usage.	Scale out, fix hot process, resize if sustained.
Memory exhaustion	Process killed or system unstable.	Use CloudWatch Agent or OS tools; EC2 basic metrics do not include memory.	Fix leak/limits, resize, or add swap only as a controlled mitigation.
EBS bottleneck	High disk latency / queue length.	Check EBS latency, IOPS, throughput, burst balance.	Tune workload or volume type/size/IOPS.
Bootstrap/user-data failure	New instance unhealthy after launch.	Inspect cloud-init/user-data logs.	Fix script, permissions, package/network dependency, or AMI mismatch.
Auto Scaling unhealthy replacement loop	Instances repeatedly launch and terminate.	Check health checks, lifecycle hooks, target-group health, bootstrap.	Fix health-check path or startup dependency before restoring desired capacity.

Useful checks / commands

```
aws ec2 describe-instance-status --instance-ids <id>
aws autoscaling describe-scaling-activities --auto-scaling-group-name <asg>
aws elbv2 describe-target-health --target-group-arn <arn>
aws ec2 get-console-output --instance-id <id> --latest
```

STAR reliability example

Scenario: An EC2-backed service began returning 503s after an Auto Scaling event.

Task: Restore healthy capacity and determine why replacement instances failed health checks.

Action: Checked ALB target health and ASG activity, then inspected instance status and cloud-init logs. Found a user-data dependency failure that prevented the service from starting. Rolled back the launch template and corrected bootstrap validation.

Resolve: Healthy targets returned, the ASG stabilized, and launch-time health validation was added.

Part IV - Cross-Service Production Failure Scenarios

Application cannot reach an internal service

- Check DNS resolution from the workload.
- If DNS works, test TCP to host:port.
- Inspect VPC route, SG/NACL, TGW/peering, firewall, and return path.
- Check IAM only if the network works but the API returns 401/403.
- Use CloudWatch/Flow Logs to verify where traffic stops.

Resolve: Restore the missing route/rule/allow-list, then validate from the actual workload.

Application gets 403 from S3

- Run `aws sts get-caller-identity` from the workload.
- Check IAM role policy, bucket policy, endpoint policy, and KMS key policy.
- Review CloudTrail for the denied action.
- Confirm bucket/key and region.

Resolve: Apply the least-privilege policy change and retest the exact S3 operation.

RDS connection timeout

- Resolve the RDS endpoint.
- Test TCP to the DB port.
- Check SG rules and subnet routing.
- Check DatabaseConnections, CPU, storage, and RDS events.
- Inspect application pool saturation.

Resolve: Fix network or DB/application capacity issue and confirm successful query.

Lambda processing backlog

- Check queue age, Lambda errors, throttles, duration, and concurrency.
- Inspect downstream latency and failed payloads.
- Check DLQ and retry behavior.

Resolve: Restore throughput, quarantine poison events, and add alarms on queue age/throttles.

EC2/ALB 503

- Check ALB target health.
- Inspect ASG activity and EC2 status checks.
- Check application process and bootstrap logs.
- Review CPU/memory/EBS pressure.

Resolve: Replace/fix unhealthy capacity and correct launch/bootstrap issue.

Part V - STAR Incident Template

Use this format when explaining an incident to a manager or interviewer. Keep the technical details, but anchor them to reliability impact.

Scenario	What was broken? Include service, environment, customer/user impact, and observable symptom.
Task	What were you responsible for? Restore service, isolate the failing layer, coordinate teams, protect data, etc.
Action	What evidence did you gather? What commands, metrics, logs, policy checks, or network tests did you use? What mitigation did you implement?
Resolve	What changed after your action? State recovery, impact reduction, validation, and prevention work.

Example manager-ready phrasing

Scenario: A production workload lost connectivity to a critical internal dependency, causing request failures. **Task:** I owned isolation of the failure across Kubernetes, AWS networking, and the downstream service. **Action:** I validated DNS and TCP from the pod, checked Route 53, VPC routes, NACL/SG rules, and flow evidence, then coordinated the peer-side allow-list fix. **Resolve:** Connectivity was restored and I added a reusable validation/runbook path to reduce future MTTR.

Part VI - Quick Production Checklist

Layer	First checks
Identity	aws sts get-caller-identity; CloudTrail; IAM/resource/KMS policies
DNS	nslookup/dig; Route 53 private hosted zones; resolver rules
TCP	nc -vz host port; security groups; NACLs; routes; firewalls
HTTP/TLS	curl -vk; status code; certificate/SNI; auth headers
EC2	status checks; ASG events; ALB health; cloud-init
S3	head-bucket/head-object; bucket/KMS/endpoint policy
RDS	connections; CPU; storage; latency; Performance Insights; events
Lambda	errors; duration; throttles; concurrency; queue age; DLQ
CloudWatch	alarms; metrics; logs; dashboards; event correlation

Production habit: Start with symptoms -> check recent changes -> establish blast radius -> validate DNS/TCP/TLS/auth/application -> mitigate -> verify -> document prevention.